Mandi and Weather Verification Dashboard

Frontend Architecture and Data Flow Guide

Audience: Project supervisor
Purpose: Explain how the verification frontend is built, how it receives data, and how the Agmarknet scraper stores data.

1. System Overview

This project has three major parts:

1. **React frontend**: The browser-based verification dashboard used by QA testers.
2. **Express backend**: The API layer between the browser and the SQL database.
3. **Python scraper**: The background process that collects mandi prices and arrival information from Agmarknet.

The browser does not connect directly to either database. It communicates only with the Express API over HTTP.

```
Agmarknet website
       |
       v
Python Playwright scraper
       |
       v
PostgreSQL: public.fact_market_prices

React frontend
       |
       | HTTP / REST API
       v
Express backend
       |
       v
Azure SQL: dbo.MarketPrices and dbo.VerificationResults
```

2. Frontend Technology Stack

Technology	Purpose
React 18	Builds the user interface with reusable components
Vite 5	Runs the development server and creates production builds
React Router 6	Handles page navigation and protected routes
Tailwind CSS 3	Provides utility-based styling and responsive layout
Lucide React	Provides interface icons

Technology	Purpose
React Hot Toast	Displays success, error, and loading notifications
JavaScript and JSX	Main frontend implementation language
Fetch API	Sends requests from the frontend to the backend

The frontend project is defined in `frontend/package.json`.

3. How the Frontend Starts

The browser entry point is `frontend/src/main.jsx`.

The startup sequence is:

1. React creates the application root.
2. `BrowserRouter` enables URL-based navigation.
3. `App.jsx` loads authentication, notifications, and application routes.
4. `AppRoutes.jsx` selects the page for the current URL.
5. Protected dashboard pages are shown only after the user has a session.

Important files:

- `frontend/src/main.jsx`: React startup entry point
- `frontend/src/App.jsx`: Global application providers and notifications
- `frontend/src/routes/AppRoutes.jsx`: Application routes
- `frontend/src/routes/ProtectedRoute.jsx`: Dashboard access protection
- `frontend/src/auth/AuthProvider.jsx`: Current client-side session handling

4. Main Frontend Pages

- `DashboardOverview.jsx`: Summary counts and coverage information
- `DataVerification.jsx`: Main data verification workflow
- `Duplicates.jsx`: Duplicate record review
- `MissingData.jsx`: Missing-data review
- `Settings.jsx`: Tester and data-source information
- `Login.jsx` and `Signup.jsx`: User entry screens

The shared layout is divided into:

- `DashboardLayout.jsx`: Combines the sidebar, header, and page content
- `Sidebar.jsx`: Main navigation
- `TopHeader.jsx`: Global search, refresh, notifications, date, and user controls

5. Data Verification Screen

The main verification screen is assembled by `DataVerification.jsx`.

It uses:

- `FilterBar.jsx`: Date, state, district, market, commodity, and status filters
- `RecordsTable.jsx`: Scraped records, prices, arrival quantity, and arrival unit
- `ImportCsvModal.jsx`: CSV review/import interface
- `useRecords.js`: Fetches records and refreshes after verification changes

The filter bar loads options through the backend endpoints:

```
GET /api/filters/states
GET /api/filters/districts
GET /api/filters/markets
GET /api/filters/commodities
```

The Status filter currently focuses on Missing and Duplicate records.

6. Frontend-to-Database Connection

The browser-side API connection is centralized in:

`frontend/src/services/verificationService.js`

It uses this API base URL:

```
import.meta.env.VITE_API_BASE || 'http://localhost:4000/api'
```

The frontend sends requests such as:

```
GET  /api/records
GET  /api/overview
GET  /api/filters/states
GET  /api/filters/districts
GET  /api/filters/markets
GET  /api/filters/commodities
GET  /api/issues
POST /api/records/:id/verify
```

The frontend never contains SQL credentials and never runs SQL queries directly.

7. Express Backend and Azure SQL

The backend is located in the `backend` folder.

- `backend/server.js`: Defines the REST API routes and converts database rows into frontend records.
- `backend/db.js`: Creates and reuses the Microsoft SQL Server connection pool.

- `backend/.env` : Stores the database connection settings and must remain private.

The backend uses the `mssql` Node.js package and reads these environment variables:

```
MSSQL_HOST
MSSQL_PORT
MSSQL_DB
MSSQL_USER
MSSQL_PASSWORD
```

The backend reads and writes these Azure SQL tables:

```
dbo.MarketPrices
 dbo.VerificationResults
```

`dbo.MarketPrices` contains scraped price records. `dbo.VerificationResults` contains the tester's review result, notes, issue type, and verification time.

The backend joins the latest verification result onto each market-price record before returning it to the frontend.

8. Scraper and Its Database Connection

The scraper is Python-based and uses Playwright to interact with the Agmarknet website.

Main files:

- `bot_processors/pricing/agmarknet_scraper.py` : Opens Agmarknet, selects filters, handles CAPTCHA, and extracts records.
- `bot_processors/pricing/scraper_daemon.py` : Runs repeated scrape cycles, retries failures, and backfills recent missing dates.
- `bot_processors/core/db_pool.py` : Creates the shared PostgreSQL connection pool.
- `db/schema.sql` : Defines the PostgreSQL price table.

The scraper reads the `DATABASE_URL` environment variable and uses the `asyncpg` package.

The scraper writes to:

```
public.fact_market_prices
```

Important fields include:

```
state, district, market, commodity, crop_keyword,
arrival_date, modal_per_kg, min_per_kg, max_per_kg,
arrival_qty, arrival_unit, scraped_at
```

`save_scraped()` in `agmarknet_scraper.py` uses an upsert. A re-scrape of the same state, district, commodity, and arrival date updates that record instead of creating a duplicate.

9. Daily Scraping Process

The normal process is:

1. The daemon starts and initializes the PostgreSQL pool.
2. Playwright opens the Agmarknet daily price and arrival report.
3. The scraper selects all states and processes each commodity.
4. Price and arrival data are extracted.
5. Results are checkpointed to `last_known_prices.json` during long runs.
6. Results are upserted into `public.fact_market_prices`.
7. The daemon checks the previous three calendar days for missing rows.
8. Missing recent dates are retried and backfilled.
9. XLSX and geographic reference files are refreshed.

The daemon is configured to run every eight hours, or three times per day.

10. Important Current Architecture Boundary

There are currently two database systems in the project:

```
Scraper:   Python -> PostgreSQL -> public.fact_market_prices
Frontend:  React -> Express -> Azure SQL -> dbo.MarketPrices
```

The frontend does not automatically read the PostgreSQL scraper table. For newly scraped records to appear in the frontend, they must be synchronized or imported into `dbo.MarketPrices`.

This separation is the main reason the scraper's latest date and the frontend's latest date can differ. It is also why scraper status and frontend database status should be monitored separately.

11. Local Run Commands

Start the frontend:

```
cd frontend
npm run dev
```

Frontend URL:

```
http://127.0.0.1:5173/
```

Start the backend:

```
cd backend
npm run dev
```

Backend URL:

```
http://localhost:4000/
```

Run the scraper daemon:

```
python -m bot_processors.pricing.scraper_daemon
```

12. Summary

The frontend is a React-based QA dashboard. It displays data through a controlled Express API, and the Express API reads Azure SQL. The Agmarknet scraper is a separate Python process that writes to PostgreSQL. Verification actions flow from the browser through Express into Azure SQL, while scraper data must be synchronized into the frontend database before it can be displayed by the dashboard.